
1. Dependency Injection (DI)

- Dependency Injection is a design pattern where objects are provided with their dependencies rather than creating them internally.
- **How it works in Spring:**
 - The Spring container (**ApplicationContext**) manages the lifecycle of beans and injects dependencies automatically.
 - Dependencies can be injected via:
 - **Constructor Injection:** Dependencies are provided through a class constructor.
 - **Setter Injection:** Dependencies are provided through setter methods.
 - **Field Injection:** Dependencies are injected directly into fields (less recommended).
- **Benefits:**
 - Promotes loose coupling.
 - Improves testability and maintainability.

2. Inversion of Control (IoC)

- Inversion of Control is a principle where the control of object creation and lifecycle is transferred to the Spring container.
- **How it works:**
 - Instead of the application creating objects, the Spring container creates and manages them.
 - The container uses configuration metadata (XML, annotations, or Java configuration) to instantiate and wire beans.
- **Benefits:**
 - Reduces boilerplate code.
 - Centralizes configuration and management of objects.

3. Key Annotations

Spring provides a variety of annotations to simplify configuration and development:

a) **@Primary**

- Indicates that a bean should be given preference when multiple beans of the same type are present.
- **Use Case:**
 - Resolving ambiguity when multiple implementations of an interface exist.

```
@Bean
@Primary
public MyService primaryService() {
    return new PrimaryService();
}
```

b) **@Controller**

- Marks a class as a Spring MVC controller.
- Handles HTTP requests and returns the appropriate response (e.g., HTML, JSON).

```
@Controller
public class MyController {
    @GetMapping("/hello")
    public String sayHello() {
        return "hello";
    }
}
```

c) @SpringBootConfiguration

- Indicates that a class provides Spring Boot configuration.
- Typically used in combination with **@EnableAutoConfiguration** and **@ComponentScan**.

```
@SpringBootConfiguration
public class MyAppConfig {
    // Configuration beans
}
```

d) @Provided

- Not a standard Spring annotation. It might refer to custom or third-party annotations used in specific contexts.
-

4. Spring Data

- Spring Data simplifies data access by providing a consistent programming model for interacting with databases.
- **Key Features:**
 - **Repository Abstraction:** Provides interfaces like **CrudRepository**, **JpaRepository**, and **PagingAndSortingRepository**.
 - **Query Methods:** Allows defining database queries using method names or **@Query** annotations.
 - **Support for Multiple Data Stores:** Works with relational databases (via JPA), NoSQL databases (e.g., MongoDB, Cassandra), and more.

```
public interface UserRepository extends JpaRepository<User, Long> {
    List<User> findByLastName(String lastName);
}
```

5. Spring Security

- Spring Security is a powerful framework for securing Spring-based applications.
- It provides authentication, authorization, and protection against common security vulnerabilities.

a) Authentication

- Verifies the identity of a user.
- **Common Mechanisms:**
 - **Form-Based Authentication:** Users log in via a form.
 - **OAuth2:** For third-party authentication.
 - **JWT (JSON Web Tokens):** For stateless authentication.
- **Example:**

```
@Override
protected void configure(AuthenticationManagerBuilder auth) throws Exception {
    auth.inMemoryAuthentication()
        .withUser("user").password("{noop}password").roles("USER");
}
```

b) Authorization

- Determines what an authenticated user is allowed to do.
- **Example:**

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http.authorizeRequests()
        .antMatchers("/admin").hasRole("ADMIN")
        .antMatchers("/user").hasRole("USER")
        .anyRequest().authenticated();
}
```

c) Filters

- Spring Security uses a chain of filters to handle security-related tasks.
- **Key Filters:**
 - **UsernamePasswordAuthenticationFilter:** Handles form-based login.
 - **BasicAuthenticationFilter:** Handles HTTP Basic Authentication.
 - **FilterSecurityInterceptor:** Enforces authorization rules.
- **Custom Filters:**
 - You can create custom filters to implement specific security logic.

```
public class CustomFilter extends OncePerRequestFilter {  
    @Override  
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, F  
    filterChain filterChain)  
        throws ServletException, IOException {  
        // Custom logic  
        filterChain.doFilter(request, response);  
    }  
}
```

Summary

- **Dependency Injection (DI):** Injects dependencies into objects.
- **Inversion of Control (IoC):** Delegates object creation and management to the Spring container.
- **Annotations:**
 - **@Primary:** Prioritizes a bean.
 - **@Controller:** Marks a class as a Spring MVC controller.
 - **@SpringBootApplication:** Provides Spring Boot configuration.
- **Spring Data:** Simplifies data access with repository abstraction.
- **Spring Security:**
 - **Authentication:** Verifies user identity.
 - **Authorization:** Controls access to resources.
 - **Filters:** Handles security-related tasks.